CORRIGÉ · POUR L'ENSEIGNANT · À NE PAS DONNER

✅ MISSION N° 1 — L'ASCENSEUR DE LA TOUR

Ceci est UNE solution, pas LA solution. L'élève a reçu un cahier des charges, pas des instructions. Tout programme qui respecte les sept règles est juste, même s'il ne ressemble en rien à celui-ci.

🎯 Pourquoi cette mission, et pour qui

Pour un élève **qui a fini avant les autres**. **Aucune notion nouvelle** : tout tient dans la boîte à outils de la séance 7. Ce qui est neuf, c'est qu'on demande **de concevoir, pas d'exécuter**. La feuille ne nomme personne : elle resservira.

- **Elle introduit l'algorithme pour de vrai.** Sa feuille donne l'exemple qui manquait, et il porte sur un code déjà écrit en classe : le coup joué de la séance 7, en français à gauche, en Python à droite, ligne pour ligne.
- **Elle force le `if` dans le `if`** : trois réponses possibles (monte / descend / déjà là) ne peuvent pas s'écrire avec un seul `if`. C'est la notion de samedi, portée un cran plus loin.
- **Sa feuille tient sur une page** et ne porte que trois choses : ce qu'est un algorithme, le cahier des charges du programme, et l'écran attendu. **Aucune étape, aucune ligne de code.** L'algorithme s'écrit **dans son cahier**, le code ensuite dans Thonny.

🧠 L'ALGORITHME MODÈLE — c'est ça qu'on lit en premier

```
1. Je cree la liste des cinq noms d'etages.
2. J'affiche la tour.
3. Je range 1 dans la boite "etage actuel".
4. Je range 0 dans la boite "etages parcourus".
5. Je range 0 dans la boite "arrets".

   -- POUR CHAQUE VOYAGE (je recopie le bloc 3 fois) --

6. Je demande l'etage, je le transforme en nombre.
7. SI cet etage est plus grand que 5 :
       je dis qu'il n'existe pas.          <-- et rien d'autre ne change
   SINON :
       SI l'etage demande est plus haut que l'etage actuel :
           je range (demande - actuel) dans "distance" ;
           je dis que ca MONTE de "distance".
       SINON :
           SI l'etage demande est plus bas :
               je range (actuel - demande) dans "distance" ;
               je dis que ca DESCEND de "distance".
           SINON :
               je range 0 dans "distance" ;
               je dis qu'on y est deja.

       j'affiche le nom de l'etage demande ;
       j'ajoute "distance" aux etages parcourus ;
       j'ajoute 1 aux arrets ;
       je range l'etage demande dans "etage actuel".

   -- FIN DU BLOC --

8. J'affiche le journal : etage actuel et son nom, etages
   parcourus, arrets sur 3.
9. SI les etages parcourus sont plus de 6 : une phrase.
   SINON : l'autre phrase.
```

🤖 Les cinq essais à lui faire faire avant de valider

Ils ne sont pas sur sa feuille, **exprès** : trouver comment on éprouve un programme fait partie du travail. Demandez-les un par un, **à la fin**.

- Monter, puis **redescendre** : le deuxième voyage part-il bien du nouvel étage ?
- Demander **l'étage où on est déjà**.
- Demander **l'étage 9** : refusé, et l'ascenseur ne bouge pas.
- Aller de **1 à 5** : le journal annonce **4 étages** parcourus, pas 5.
- Rater un voyage : le journal dit **2 / 3**.

Et si tout est fini, trois prolongements à donner oralement : compter les voyages **refusés** ; retenir **l'étage le plus haut** atteint ; afficher **une ligne par étage traversé** pendant un trajet. **Le troisième est un piège volontaire** : il demande de répéter un nombre de fois qu'on ne connaît pas d'avance, et l'outil n'arrive que le 14 novembre. Demandez l'algorithme quand même, et gardez-le pour ce jour-là.

💬 Comment l'accompagner sans lui donner la réponse

La règle : on ne répond jamais par une ligne de code. On répond **par une question**, et on s'en va.

- « Montre-moi ton algorithme. » *(Pas d'algorithme écrit, pas de code : on s'arrête là, retour à la feuille. C'est la seule consigne ferme de la mission.)*
- « Lis-moi la phrase 7 à voix haute. » *Entendre son propre algorithme révèle les trous mieux que n'importe quelle relecture.*
- « Fais-le à la main : tu es à l'étage 2, tu demandes le 5. Qu'est-ce qui change ? » *Pour la boîte oubliée.*
- « Combien de réponses différentes ton programme peut-il donner ici ? » *Pour amener au `if` dans le `if` sans le nommer.* — et « Essaie le 9. » *plutôt que de signaler l'oubli du refus.*

✂️ **Le début** (la liste des cinq noms, l'affichage de la tour, les trois boîtes de départ) **est sans surprise** : il est dans le fichier `mission-1-ascenseur-corrige.py`. Voici **le bloc du voyage**, le seul qui compte — il est **recopié trois fois**, et le journal de bord vient après.

```
# ======================================================================
#  VOYAGE 1   (c'est ce bloc que l'eleve recopiera trois fois)
# ======================================================================
demande = input("Voyage 1 - a quel etage vas-tu ? ")
demande = int(demande)

if demande > 5:
    print("   Cet etage n'existe pas. La tour va de 1 a 5.")
else:
    if demande > actuel:
        distance = demande - actuel
        print("   L'ascenseur MONTE de " + str(distance) + " etage(s).")
    else:
        if demande < actuel:
            distance = actuel - demande
            print("   L'ascenseur DESCEND de " + str(distance) + " etage(s).")
        else:
            distance = 0
            print("   Tu y es deja ! Les portes se rouvrent.")

    #  Ces quatre lignes sont DANS le else du premier if, mais
    #  DEHORS des trois branches : elles servent dans les trois cas.
    print("   Etage " + str(demande) + " : " + etages[demande - 1])
    parcourus = parcourus + distance
    arrets = arrets + 1
    actuel = demande

print("")
```

❌ Ce qui est vraiment faux — et ce qui ne l'est pas

Vraiment faux	Juste autrement — on ne corrige pas
`actuel = demande` **oublié** — l'ascenseur ne bouge jamais. **À chercher en premier.**	D'autres noms d'étages, d'autres messages, une tour affichée dans l'autre sens.
Les quatre lignes de mise à jour **placées dans une seule branche** au lieu d'être dessous : l'ascenseur ne se met à jour qu'un cas sur trois.	Recalculer la distance dans chaque branche au lieu de garder une boîte `distance` : plus long, pas plus faux.
`etages[demande]` sans le `- 1` : le nom affiché est celui de l'étage du dessus, **et Python ne dit rien**.	`demande > 5` ou `demande >= 6`.
Un étage inexistant accepté (`IndexError` ou un nom faux), ou `parcourus = parcourus + distance` écrit **avant** le calcul de la distance.	Compter aussi les voyages ratés, ou tenir un journal plus riche que demandé : c'est un bonus, pas une faute.

Le zéro et les nombres négatifs ne sont pas au programme de cette mission — les refuser demanderait le `and` de la séance 9. Si l'élève le remarque, c'est un excellent signe : dites-lui que c'est exactement la question du 7 novembre.

✅ Comment l'évaluer — par la preuve, pas par une note

Ce qu'on regarde	La preuve
1. L'algorithme existe	Il est dans le cahier, en français, avec des **SI** décalés. **C'est le vrai livrable de la mission.**
2. Joué à la main	L'élève sait dire, sans la machine, ce que devient chaque boîte quand on va de l'étage 2 au 5.
3. Le programme tourne	Les trois voyages s'enchaînent sans message rouge.
4. Il tient debout	Les **cinq essais** listés plus haut passent, le 9 refusé compris.
5. Expliqué	L'élève sait dire **pourquoi** les quatre lignes de mise à jour sont à ce niveau de décalage, et pas plus à droite.

Les points 1 et 5 comptent plus que le 3 — ce n'est pas écrit sur sa feuille : **dites-le vous-même, avant le début. Et après :** faites-lui présenter **son algorithme** à la classe en trois minutes, au tableau, **sans montrer le code**. Pour les autres, c'est le premier algorithme qu'ils verront écrit par quelqu'un de leur âge.